# Rainfall across Singapore, 2017–2024: patterns and a 30-minute rain model with Apache Spark

DSA5208 Project 2

Kushagra Bhatnagar (A0351591A) · Pratyush Sinha (A0352482A)

All numbers and tables produced from the pipeline outputs by `src/report_macros.py`, 2026-10-17 20:32 +08

**Summary.** We used all 8 annual files of the data.gov.sg "Historical Rainfall across Singapore" collection, 2017 to 2024: 48,538,713 five-minute readings from 91 stations, processed and modelled with Apache Spark (DataFrame API and MLlib). In Task 1, November is the wettest month and July the driest. Rain peaks in the afternoon in every season except the south-west monsoon, when it peaks in the morning. Wet and dry years differ in how often it rains, not how hard: 2019 was dry because fewer slots had rain, while intensity barely moved. In Task 2 we model $P(\text{rain in the next 30 min} > \theta)$ from the station, the time and $\theta$. A strict model that uses only these three inputs does no better than climatology (test AUC-PR 0.084 against 0.085). A nowcasting model with the same interface, which also looks up the rain observed up to the prediction time, reaches AUC-PR 0.654 and Brier skill 0.453 on 2024, a year we kept aside for a single final test, and its probabilities are close to calibrated. Against the official Meteorological Service Singapore (MSS) daily records, our data agree closely on complete days (correlation 0.991 over 88,443 station-days). The gaps, however, are not random: a network outage on 1–4 January 2021 hides some of the heaviest rain of Singapore's wettest January on record.

## 1 Data and cleaning

### 1.1 Data and ingestion

Each annual file has one row per station and 5-minute interval: the time, the station (ID, name, coordinates) and the rain in mm (`reading_value`). A reading is the rain in the five minutes ending at its timestamp. All times are Singapore time, and we set Spark's session time zone to match.

We read the eight CSVs with Spark using an explicit schema instead of schema inference, so that a malformed line would be kept in a corrupt-record column rather than dropped without trace, and wrote them to Parquet partitioned by year (`src/ingest.py`). To confirm that nothing was lost on the way, `checks/ingest_check.py` compares, for each year, the Parquet row count with the number of data lines in the CSV, and stops the pipeline if a year is missing or a count differs. Table 1 is its output: every year is present and matches line for line. The network is not fixed: it has between 54 and 80 stations depending on the year, and the 2017 file starts only at 08:05 on 1 January.

| Year | Parquet rows | CSV data lines | Stations | First reading (SGT) | Last reading (SGT) |
|---|---|---|---|---|---|
| 2017 | 5,256,106 | 5,256,106 | 62 | 2017-01-01 08:04:59 | 2017-12-31 23:55:00 |
| 2018 | 4,594,246 | 4,594,246 | 56 | 2018-01-01 00:00:00 | 2018-12-31 23:55:00 |
| 2019 | 5,026,862 | 5,026,862 | 54 | 2019-01-01 00:00:00 | 2019-12-31 23:55:00 |
| 2020 | 6,334,473 | 6,334,473 | 80 | 2020-01-01 00:00:00 | 2020-12-31 23:55:00 |
| 2021 | 7,044,646 | 7,044,646 | 76 | 2021-01-01 00:00:00 | 2021-12-31 23:55:00 |
| 2022 | 7,133,230 | 7,133,230 | 72 | 2022-01-01 00:00:00 | 2022-12-31 23:55:00 |
| 2023 | 6,742,890 | 6,742,890 | 74 | 2023-01-01 00:00:00 | 2023-12-31 23:55:00 |
| 2024 | 6,406,260 | 6,406,260 | 70 | 2024-01-01 00:00:00 | 2024-12-31 23:55:00 |

**Table 1:** Ingestion, one row per annual file: rows written to Parquet against data lines in the CSV, number of stations, and the first and last reading (Singapore time). The two counts are equal in every year. *Made by* `checks/ingest_check.py`.

### 1.2 Cleaning

The cleaning stage (`src/clean.py`) applies the rules of Table 2 in order, each to the rows the previous rule left, so the removals add up exactly to the difference between input and output. `checks/clean_ledger.py` then recounts both from the Parquet files and stops the pipeline if a single row is unaccounted for. In the narrow sense the data were clean: there were no corrupt, missing, negative or implausible values (more than 50 mm in five minutes). Two problems needed a rule:

- 1,775,327 timestamps were one second before a five-minute mark (the first 2017 reading, for example, is stamped 08:04:59). We moved them to the nearest slot (largest shift 1 s); nothing was removed.
- 86 readings repeated a (station, slot) pair. We kept the latest update; none of the duplicates had a conflicting value (0 conflicts).

The duplicates were the only rows removed: 48,538,713 rows went in and 48,538,627 came out. We also looked for "stuck" gauges, that is, long runs of the same positive value. The longest run was 35 slots of 0.2 mm (S07, 2018). That is plausible drizzle, so we added no rule for it.

| Rule | Removed | Of | What it removes |
|---|---|---|---|
| corrupt | 0 | 48,538,713 | line not parseable with the explicit schema |
| missing | 0 | 48,538,713 | station, timestamp or value missing |
| type_unit | 0 | 48,538,713 | not a 5-minute rainfall total in mm |
| bad_ts | 0 | 48,538,713 | timestamp not ISO 8601 with offset |
| year_mismatch | 0 | 48,538,713 | timestamp outside the file's calendar year (SGT) |
| negative | 0 | 48,538,713 | negative rainfall |
| implausible | 0 | 48,538,713 | more than 50 mm in 5 minutes |
| snap | 0 (1,775,327 adjusted) | 48,538,713 | timestamp moved to the nearest 5-minute slot end |
| snap_year_cross | 0 | 48,538,713 | snapping moved the slot into another year |
| duplicate | 86 | 48,538,713 | second or later reading for the same station and slot |

**Table 2:** Cleaning ledger, rules in the order applied: rows removed by each rule and the number of rows it saw. Only the duplicate rule removed rows; the timestamp snap adjusted rows without removing any. *Made by* `src/clean.py`; `checked by checks/clean_ledger.py`.

Cleaning cannot fill gaps, and the data have many. Table 3 shows how much of each year was reported: between 75 % and 94 % of the possible station-slots. Some months are almost empty for the whole network. August 2018 has data on 3 days, February 2018 on 21 and January 2023 on 23. Because both the station set and the coverage change from year to year, we never compare raw sums across years; every rate in this report is computed per reported slot (Section 2.1).

| Year | Stations | Station-days | Days $\geq$ 90 % slots | Slots reported | Days with data |
|---|---|---|---|---|---|
| 2017 | 62 | 19,251 | 89.1 % | 80.6 % | 365 |
| 2018 | 56 | 17,356 | 79.3 % | 78.0 % | 326 |
| 2019 | 54 | 18,055 | 93.5 % | 88.6 % | 354 |
| 2020 | 80 | 23,350 | 91.6 % | 75.1 % | 364 |
| 2021 | 76 | 24,878 | 97.0 % | 88.2 % | 362 |
| 2022 | 72 | 25,129 | 97.1 % | 94.2 % | 365 |
| 2023 | 74 | 23,765 | 96.7 % | 86.7 % | 357 |
| 2024 | 70 | 22,566 | 97.1 % | 86.8 % | 366 |

**Table 3:** Coverage per year: stations, station-days (a station-day counts if the station reported at least once that day), share of those days with at least 90 % of slots, share of all possible station-slots reported, and days on which any station reported. 2018 has the fewest days with data and 2020 the lowest share of slots. *Made by* `checks/coverage_check.py`.

The measurement resolution also changed. Until 2019 almost every positive reading is a multiple of 0.2 mm, the tipping-bucket step: 100 % of them in 2017 and 98 % in 2019. From 2020 values such as 0.197 mm appear, and only 66 % (2021) to 68 % (2024) of positive readings are multiples of the step. We read this as a change in instruments or processing. We did not test whether it moved the across-year results: the fixed station panel (Section 2.1) holds the stations constant, not the instruments, so the change stays a limitation of this report.

### 1.3 Checking the data against the official MSS daily records

data.gov.sg warns that the data "may contain missing records and has not undergone quality control". We therefore compared them with the official MSS daily station records (`weather.gov.sg`). These serve only as a check: neither task uses MSS values. `src/mss_fetch.py` downloaded the daily files one at a time, with a

| Station | Month | Official (mm) | Ours (mm) | Days missing | Official on them (mm) | Share | of which network outage |
|---|---|---|---|---|---|---|---|
| S64 | Aug 2022 | 406.4 | 0.0 | 29 | 402.0 | 99 % | 0.0 |
| S24 | Jan 2023 | 302.6 | 0.0 | 30 | 302.6 | 100 % | 12.6 |
| S07 | Jan 2021 | 290.8 | 0.0 | 27 | 290.8 | 100 % | 0.0 |
| S108 | Nov 2024 | 243.2 | 11.8 | 25 | 231.4 | 95 % | 0.0 |
| S24 | Jan 2021 | 692.8 | 341.6 | 3 | 216.2 | 31 % | 216.2 |
| S122 | Jan 2021 | 204.2 | 7.8 | 17 | 196.4 | 96 % | 0.0 |
| S106 | Jan 2021 | 435.8 | 123.4 | 3 | 190.6 | 44 % | 190.6 |
| S94 | Jan 2021 | 630.2 | 324.6 | 3 | 190.4 | 30 % | 190.4 |
| S84 | Jan 2021 | 618.4 | 290.8 | 3 | 189.2 | 31 % | 189.2 |
| S78 | Jan 2021 | 521.8 | 220.4 | 3 | 179.6 | 34 % | 179.6 |

**Table 4:** Station-months with the most official MSS rain on days missing from our data, counting only months in which the station reported at least once. Totals in mm. "Days missing" counts days with no reading at all; a day with some readings (such as 1 January 2021 before 07:55) is a partial day. "Share" is the missing rain as a share of the month's official total, and the last column is the part of it that fell on network-outage days. In the worst months a station lost nearly its whole month. *Made by* `src/mss_validate.py`.

pause between requests, and cached them. 54 of our station IDs have MSS records, giving 4729 station-month files. `src/mss_validate.py` sums our readings into local calendar days and joins them to the MSS daily totals.

Where both exist, they agree. On the 88,443 station-days on which our station reported all 288 slots, our daily total and the official one correlate at 0.991, with a mean absolute difference of 0.086 mm and a mean difference of −0.061 mm. On 99.1 % of these days the two agree to within 0.2 mm, and over all of them our rain adds up to 0.991 times the official total. Large disagreements are rare (101 days differ by more than 20 mm), and the largest look like faulty gauges, in both directions. On 2021-04-01 station S36 reports 318.2 mm spread over the whole day, while MSS records 0.0 mm and no nearby station in our data reports heavy rain. On 2023-12-25, during a storm, station S104 reports only 0.6 mm across all its slots, while MSS records 134.0 mm.

None of our cleaning rules can catch days like these, because every value is plausible on its own (Figure 1a). For the extremes they matter little. On complete matched days we count 2,271 days with at least 50 mm against 2,305 in the official records; only 2 of ours had less than 5 mm officially, and only 13 official heavy days show less than 5 mm in our data.

Agreement on complete days says nothing about the days we lack. To measure what the gaps cost, we put every station-day that has an official record into one of five classes and computed the share of official rain in each:

- *complete* (all slots): 66.1 % of official rain;
- *partial* (some slots missing): 23.5 %;
- *network outage* (no station in our data reported that day): 1.7 %;
- *station gap* (the station is silent that day but reported in the same month): 1.0 %;
- *station absent* (the station has no reading in the whole month): 7.7 %.

The last class reflects which stations were in the network, not missing data. The other gaps are not random. The wettest network-outage day, 2021-01-02, averaged 129.5 mm over the 40 MSS stations, with up to 210.6 mm at one of them. MSS reported January 2021 as Singapore's wettest January on record. At Changi (S24) its official total for that month is 692.8 mm; our dataset has 341.6 mm. Our data stop at 07:55 on 1 January and resume on 5 January, and that gap covers the days on which MSS recorded 134.4 mm (1 January) and 210.6 mm (2 January) at Changi. Table 4 lists the station-months that lose the most official rain to missing days. In some of them a single station was silent; others fall in the January 2021 outage.

## 2 Investigating weather patterns and developing the model

### 2.1 Task 1: how we compared months and years

All aggregation runs in Spark (`src/task1.py`); only the small result tables go to Python for plotting. Because the stations and the coverage change, the basic quantity is a *rate* rather than a sum: the mean five-minute reading over reported slots, times 288, that is, mm per station-day. Missing slots do not lower a month's rate, and the number of stations in a year does not enter it. For a year we use a *month-balanced* total, the

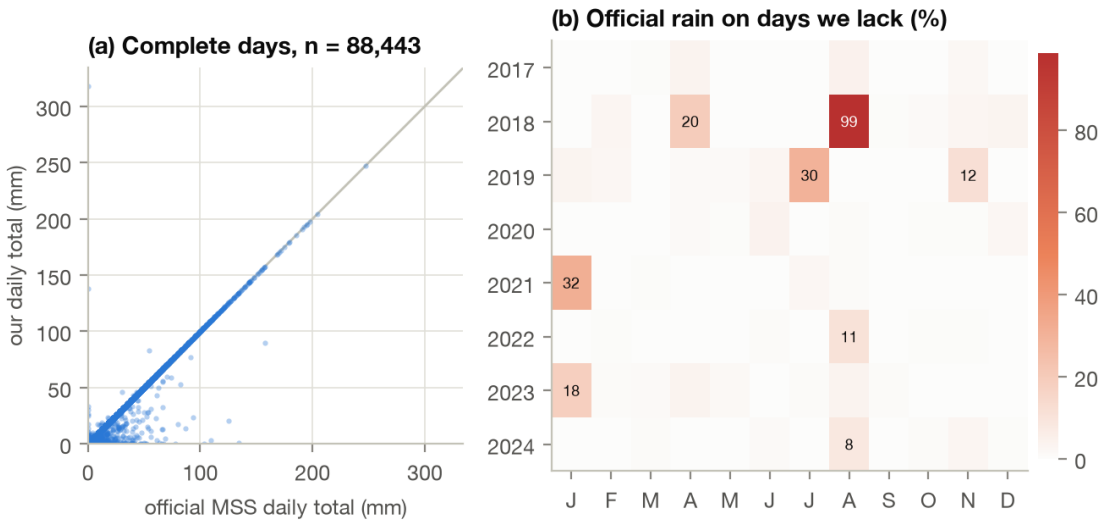



**Figure 1:** (a) Daily rain on the 88,443 complete station-days, our total against the official MSS total (mm). Nearly all points lie on the diagonal; the few far below or above it are the faulty-gauge days described in the text. (b) Share of official rain, per year and month, that fell on days our data lack (stations reporting that month). The losses are concentrated in a few months, above all August 2018, then January 2021 and July 2019. *Made by* `src/plot_mss.py from outputs/mss/`.

sum over the twelve months of (monthly rate × days in the month), so that a missing week in a wet month cannot lower the year.

We split each rate into *frequency*, the share of slots with rain, and *intensity*, the mean rain per wet slot in mm/h. This split is what lets us say whether a dry year had fewer showers or weaker ones. Extremes use complete station-hours only (all twelve slots) and complete station-days (at least 90 % of slots). Seasons follow the MSS monsoon calendar: north-east monsoon December–March, inter-monsoon April–May, south-west monsoon June–September, inter-monsoon October–November.

Two checks test whether these comparisons hold. A *fixed panel* of the 32 stations that reported at least half of every year removes the effect of stations joining and leaving the network, so a pattern that is real should appear in the panel too. The MSS comparison of Section 1.3, applied per month, finds months whose rate is distorted because the missing days were not ordinary days.

### 2.2 Task 2: target and thresholds

For station $s$, prediction time $t$ (the end of a five-minute slot) and threshold $\theta$, the label is $y = 1$ if the station's rain in $(t, t + 30\,\text{min}]$, the sum $f_{30}$ of the next six readings, exceeds $\theta$. We create a label only when all six future slots were reported and lie in the same calendar year as $t$. A missing reading therefore never counts as no rain, and no label crosses a split boundary. Because $\theta$ is an input, one model answers every threshold.

We chose the thresholds from the training years alone (Figure 2). Wet 30-minute totals have a median of 0.6 mm, a ninetieth percentile of 7.0 mm and a ninety-ninth percentile of 24.6 mm. Measurable rain (more than 0 mm) follows 6.00 % of prediction times, and more than 20 mm follows 0.11 %, which still gives 37,580 cases in six years. Above that the cases run out quickly: 10,282 above 30 mm and 2,490 above 40 mm. At the low end, the gauge step of 0.2 mm makes "more than 0.2 mm" almost the same event as "more than 0 mm", so we left that threshold out. The grid is therefore $\theta \in \{0, 0.5, 1, 2, 5, 10, 20\}$ mm, from any measurable rain to a heavy downpour.

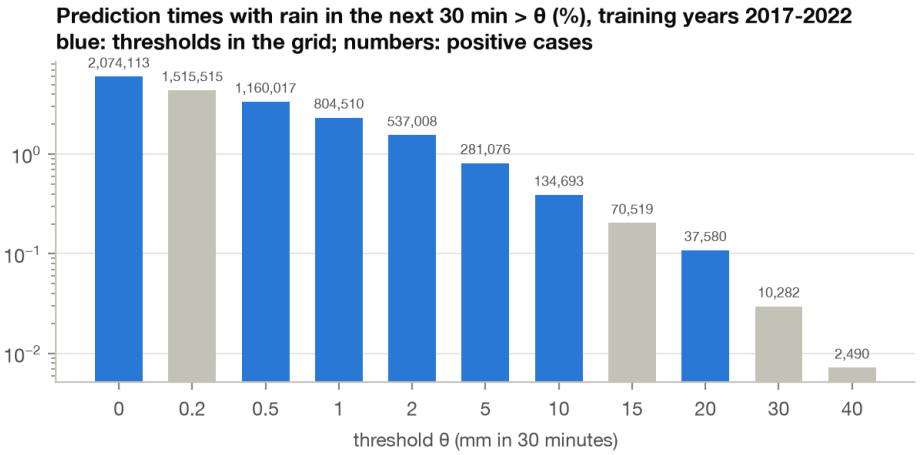



**Figure 2:** Share of prediction times in the training years 2017–2022 followed by more than $\theta$ mm of rain in the next 30 minutes (log scale), with the number of positive cases above each bar. Blue bars are the thresholds we kept. The share falls steeply with $\theta$, and beyond 20 mm few cases remain. *Made by* `src/plot_task2.py from src/task2_features.py`.

### 2.3 Task 2: features

The brief says the model should take the station, prediction time and threshold as inputs, but not whether it may also look at observations available at the prediction time. We built both readings behind the same interface, predict$(s, t, \theta)$ in `src/predict.py`, and report both.

*Model A* (strict) uses only functions of the three inputs, which makes it climatology learned by MLlib:

- the station: one-hot index, coordinates, and its wet-slot rate in the training years;
- the time: time of day and day of year as sine/cosine pairs, so that the end of a day sits next to the start of the next, plus month, hour and monsoon season;
- the threshold: $\theta$, $\log(1 + \theta)$ and a one-hot $\theta$ class.

*Model B* (nowcast) keeps all of A's features and, inside predict, also looks up what was observed up to $t$:

- the station's own rain in the last 5, 15, 30, 60 and 120 minutes and 24 hours, the minutes since its last wet slot, and how many of the last 24 slots were reported;
- at the same slot, the maximum, mean and share wet of the 6 nearest stations;
- the island-wide share of wet stations, and the maximum and mean 30-minute totals;
- the difference between the rain of the last 30 minutes and $\theta$.

Every B feature uses only readings whose slot ends at or before $t$. We computed them with Spark window functions over each station's readings ordered by slot, using `rangeBetween` on the slot index so that a missing slot is simply absent from the window instead of shifting it (`src/task2_features.py`).

### 2.4 Task 2: model, split and evaluation

We split by time: training on 2017–2022, model choice on 2023, and a single test on 2024. A random split (the `randomSplit` of Lecture 6) would put neighbouring five-minute rows of the same storm into both training and test and so overstate the test score.

Before fitting any model we froze the evaluation plan. The split, threshold grid, sampling, features, candidate models, metrics, baselines and selection rule are written in `config/eval_plan.json`, and we recorded the file's SHA-256 hash (`fd017e8e1abb...`) at 2026-10-17 14:34. The training and test scripts refuse to run if the file has changed since then, and the test script refuses to run a second time (`checks/eval_plan.py`). No result on 2024 could therefore feed back into a modelling choice.

Most prediction times are followed by no rain at all (Figure 2), so we sample them. The sample is deterministic, drawn by a hash of station, slot and seed, and has two strata: *active* times, with rain at the station in the last 2 hours or the next 30 minutes, are sampled at rate 0.15 in training, and the rest at 0.006. Every row is weighted by the inverse of its sampling rate, in fitting and in every metric, so all results refer to all prediction times and not only to the sample. Each training row is paired with one $\theta$ from the grid, while each validation and test row is scored at every $\theta$. This gives 871,046 training rows, 4,166,211 validation rows and 3,899,441 test rows.

Each candidate is an MLlib `Pipeline`: `SQLTransformer` for the derived inputs, `StringIndexer` for the station, `OneHotEncoder` and `VectorAssembler`, and then `LogisticRegression` or `GBTClassifier` fitted with `weightCol`. The saved `PipelineModel` therefore takes the raw columns. For each of A and B the candidates were logistic regression at two regularisation strengths and gradient-boosted trees, and the plan fixed the selection rule in advance: the lowest weighted log loss on 2023, pooled over the grid.

We compare the models with two baselines, both estimated on the full, unsampled training years. *Climatology* is the frequency of $y$ per (station, month, hour, $\theta$), shrunk toward the (month, hour, $\theta$) frequency with 20 pseudo-counts. *Persistence* is the frequency of $y$ given whether the last 30 minutes at the station exceeded $\theta$.

AUC-ROC and AUC-PR come from MLlib's `BinaryClassificationEvaluator`, with `weightCol`. The other metrics are Spark aggregations: Brier score, Brier skill against the base rate, log loss and a 10-bin reliability table. We report every metric pooled over the grid and per $\theta$. We also count how often the predicted probability *rises* with $\theta$, which should never happen, since a higher threshold is a rarer event. We read AUC-PR first because the events are rare: a model with no skill scores the base rate, not one half.

## 3 Computing environment

### 3.1 Local machine

All reported numbers were computed on our local machine. Every Spark job writes an `env.json` file when it starts (`checks/env_capture.py`); the specification below and Table 5 are read from those files.

Apple M5 with 10 logical CPUs and 16 GiB of memory, running macOS 26.5.1. Apache Spark 4.0.4 (PySpark) on Java 17.0.20.1 and Python 3.11.17, in local mode, master `local[10]`: one JVM whose driver runs 10 task threads, one per logical CPU, with driver memory 10g, 64 shuffle partitions and session time zone Asia/Singapore. In local mode the driver also does the executors' work, so the driver memory is the memory Spark has. The recorded stages took about 17 minutes of wall time in total.

| Stage | Run id (`runs/<id>/env.json`) | Wall time (s) |
|---|---|---|
| ingest (CSV → Parquet) | 20261017T152317-ingest | 23 |
| ingest check | 20261017T152343-ingest_check | 19 |
| cleaning + ledger | 20261017T152406-clean | 625 |
| ledger check | 20261017T153435-clean_ledger | 14 |
| coverage | 20261017T153452-coverage_check | 4 |
| stuck-gauge check | 20261017T153458-stuck_runs | 11 |
| Task 1 aggregation | 20261017T153512-task1 | 63 |
| Task 2 features | 20261017T153619-task2_features | 103 |
| Task 2 samples + baselines | 20261017T153805-task2_sample | 42 |
| Task 2 fit + validation | 20261017T143824-task2_train | not recorded[†] |
| Task 2 test (once) | 20261017T150558-task2_test | 83 |
| MSS validation | 20261017T154130-mss_validate | 61 |

**Table 5:** Local runs behind the reported outputs: one row per pipeline stage, with the run identifier (its `runs/<id>/env.json` holds the full environment) and wall time in seconds. Of the recorded stages, cleaning takes the longest. [†] The fit-and-validation run started before we added wall-time capture; its per-model fit times are in Table 8. *Made by* `src/report_macros.py from runs/*/env.json`.

### 3.2 Google Cloud Dataproc

We also ran the pipeline once on Google Cloud Dataproc, from ingestion to the Task 2 fit and validation. The 2024 test and every number reported elsewhere come from the local runs. Cluster `dsa5208-p2`, zone `asia-southeast1-c`, image `3.0.3-debian13`: 1 n2-standard-4 master and 2 n2-standard-4 workers, each with a boot disk of 80 GB pd-balanced; Spark 4.1.2 on Java 21.0.12.1 and Python 3.12.13, master `yarn`; 2 executors with 3 cores and 8g each, driver memory 8g, 48 shuffle partitions, dynamic allocation off.

These three machines took the project's whole vCPU quota, so the cluster could not be larger. We sized the executors as Lecture 6 does. Each machine has 4 vCPUs and 16 GiB of memory. Lecture 6 recommends three to five cores per executor and keeping some capacity on each worker for the cluster manager and the operating system, so we ran one executor per worker with 3 cores and left the remaining vCPU free. Executor memory, plus an overhead that Lecture 6 puts at about a tenth of it (more for PySpark), has

to fit within what YARN offers on a worker, which is less than the machine's 16 GiB; 8g fits with room to spare for the Python worker processes. We turned dynamic allocation off so that every job ran on exactly these executors. The raw CSVs were downloaded from data.gov.sg straight into Cloud Storage by a job on the cluster (`cloud/fetch_raw_on_cluster.py`) rather than uploaded from our local machine. The run used two clusters of this configuration. The stages up to the cleaning-ledger check ran on the first (`cloud_run/cluster_part1.json`). A status call on our side hung, the cluster sat idle until its idle limit deleted it, and we resumed the remaining stages on a second cluster, which read the first cluster's outputs from the same Cloud Storage prefix.

Table 6 compares the wall time of each stage; the cloud stages took about 44 minutes in total. Every Dataproc job pays a fixed start-up cost (the trivial environment-check job took 29 s), and on the short stages that cost dominates. Our local machine also has 10 cores against the cluster's 6 executor cores, and reads local disk rather than Cloud Storage. Cleaning, the longest data stage, is where the cluster catches up.

To check that the cluster computed the same results, `checks/cloud_parity.py` compares its outputs with the local ones (22 tables). Counts that do not depend on the order of summation (rows, stations, every cleaning rule, coverage, slot and event counts, sample sizes) match exactly; across 34 floating-point columns the largest relative difference is $3.1 \times 10^{-13}$. The 4 threshold counts on summed rain (hours with at least 20 or 40 mm, days with at least 50 mm) differ by at most 4 per row. Most readings are multiples of 0.2 mm, so many totals equal a threshold exactly, and a floating-point sum lands just above or below it depending on the order in which Spark adds partitions. `checks/threshold_boundary.py` counts the totals within $10^{-6}$ mm of each threshold, and all differences are within that count. The 5 approximate quantiles (Spark's `percentile_approx`) differ by up to 1.3 %. On 2023 the Task 2 fit on the cluster chose the same models as the local fit; Model B's validation AUC-PR is 0.6677 on Dataproc and 0.6677 locally, and the largest AUC-PR difference over all candidates is 0.0023.

| Stage | Local (s) | Dataproc (s) | Dataproc / local |
|---|---|---|---|
| ingest (CSV → Parquet) | 23 | 156 | 6.7 |
| ingest check | 19 | 45 | 2.3 |
| cleaning + ledger | 625 | 580 | 0.9 |
| ledger check | 14 | 48 | 3.4 |
| coverage | 4 | 51 | 13.4 |
| Task 1 aggregation | 63 | 143 | 2.3 |
| Task 2 features | 103 | 255 | 2.5 |
| Task 2 samples + baselines | 42 | 133 | 3.2 |
| Task 2 fit + validation | not recorded | 1251 | – |

**Table 6:** Wall time per stage in seconds, on the local machine and on Dataproc, and their ratio; a ratio below one means Dataproc was faster. *Made by* `src/report_macros.py from runs/*/env.json and cloud_run/job_times.json (Dataproc jobs API; driver start to exit, including the small Cloud Storage copies)`.

## 4 Results

### 4.1 Task 1: patterns within a year

The year has two wet peaks and two dry spells (Figures 3 and 4). Over 2017–2024, November is the wettest month at 10.5 mm per station-day and July the driest at 5.0. After November come December (8.5), April (8.3) and January (7.9); February (5.3) and July–August (5.0, 6.2) are the dry spells. Grouped by monsoon season, the October–November inter-monsoon is the wettest at 8.6 mm/day and the south-west monsoon the driest at 6.5, with the north-east monsoon (7.2) and the April–May inter-monsoon (7.6) in between. These are eight-year means, and single years scatter widely around them: the same month can be three times as wet in one year as in another.

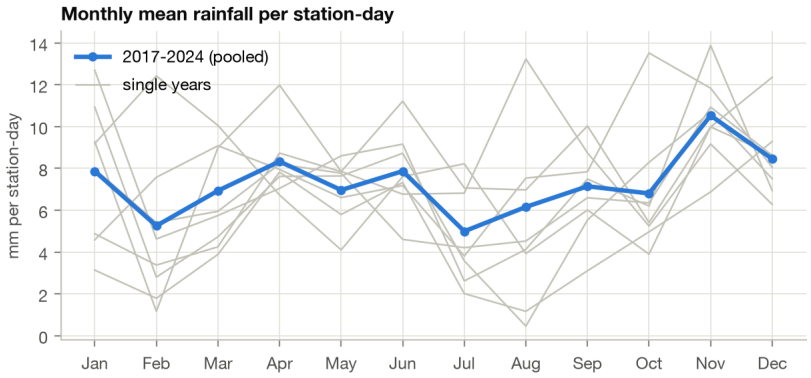



**Figure 3:** Mean rainfall per station-day (mm) in each calendar month, pooled over 2017–2024 (blue) and for each single year (grey). The pooled cycle peaks in November and April and dips in February and July; single years spread far more widely than the pooled months differ from each other. *Made by* `src/plot_task1.py from src/task1.py`.

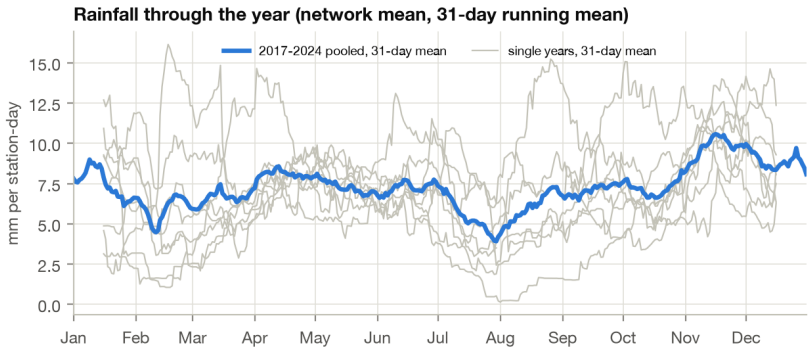



**Figure 4:** Network rainfall (mm per station-day) through the calendar year as a 31-day running mean (total rain over reported slots in the window), pooled over 2017–2024 (blue) and for each year (grey). The two dry spells fall in mid-February and around the turn of July and August. *Made by* `src/plot_task1.py from src/task1.py`.

Within the day, the timing depends on the season (Figure 5). In the north-east monsoon and both inter-monsoons, rain peaks in the mid-afternoon, at 16:00–17:00, 15:00–16:00 and 15:00–16:00 SGT respectively, with the highest rate (1.14 mm/h) in October–November. Overnight the rate falls to about 0.10 mm/h. We read this as convective storms building over land in the afternoon heat. The south-west monsoon peaks instead at 09:00–10:00 (0.58 mm/h, against 0.07 at its lowest hour), when squall lines that form over Sumatra overnight reach Singapore. Over all seasons, 55 % of the rain falls between 12:00 and 20:00.

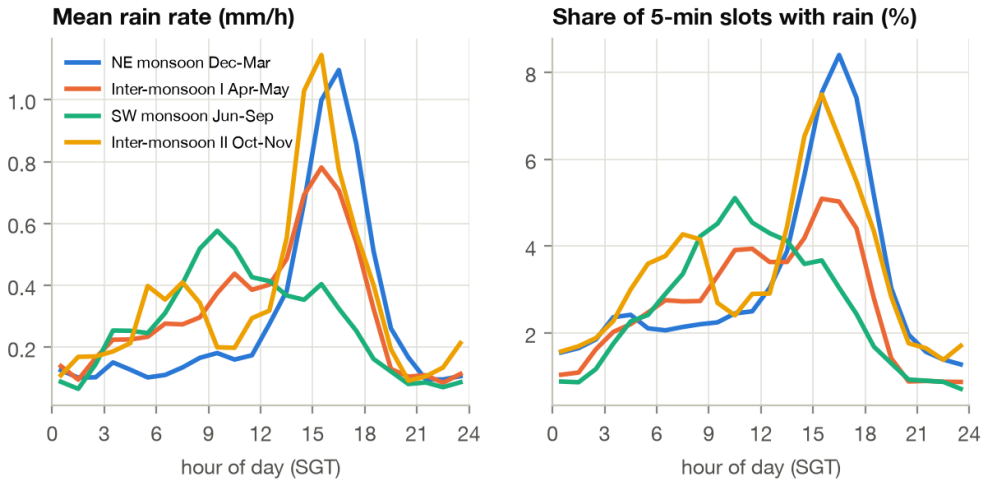



**Figure 5:** Rain by hour of day (SGT) in each monsoon season, 2017–2024: mean rate in mm/h (left) and share of five-minute slots with rain (right). Three seasons peak in the mid-afternoon; the south-west monsoon (green) peaks at 09:00–10:00 instead. *Made by* `src/plot_task1.py from src/task1.py`.

A month's rate is the frequency of rain times its intensity, and the two wet peaks are built differently (Figure 6). Over the whole period it rains in only 2.9 % of slots, yet 47 % of all rain falls at 30 mm/h or more. January departs from this in one direction: it has the most wet slots (4.4 %) but the gentlest rain, 7.5 mm/h when wet, and only 34 % of its rain falls at 30 mm/h or more. That is long, steady monsoon rain. April departs in the other: its wet slots are close to the all-period share (2.8 %), but it has the hardest rain (12.3 mm/h, with 53 % of it at 30 mm/h or more), which points to short thunderstorms.

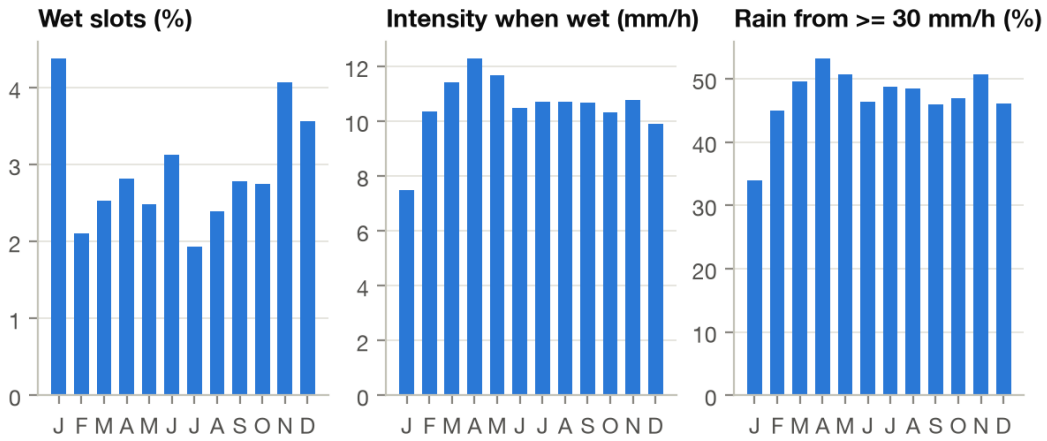



**Figure 6:** Each calendar month, 2017–2024: share of five-minute slots with rain (%), mean intensity of wet slots (mm/h), and share of the month's rain that fell in slots of at least 30 mm/h (%). January is wet most often but most gently; April has the hardest rain. *Made by* `src/plot_task1.py from src/task1.py`.

Across the island, stations further west tend to be wetter, in every season (Figure 7). The correlation of a station's mean rate with its longitude is −0.27 in the north-east monsoon and −0.62 in the south-west monsoon, the season whose squalls arrive from the west. Over the 83 stations with at least a year of data, the mean rate runs from 4.8 mm/day at S208 to 9.8 mm/day at S228.

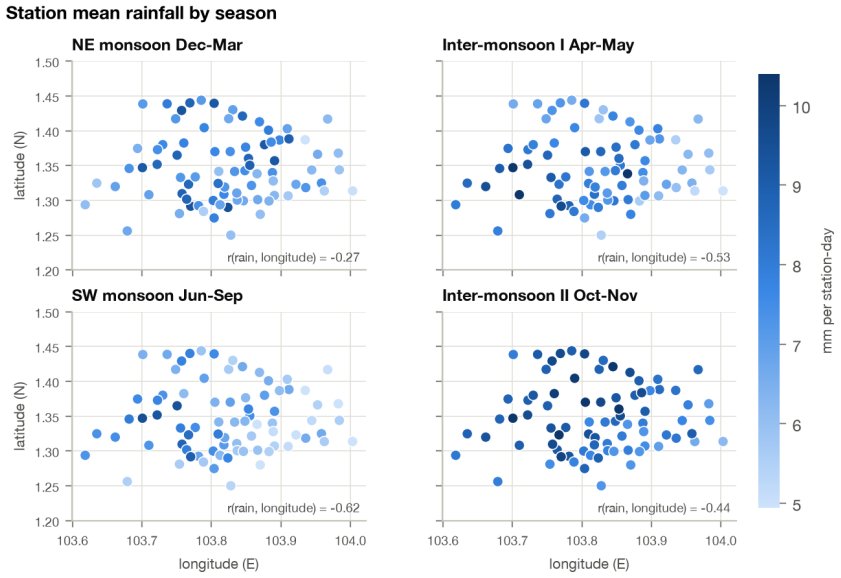



**Figure 7:** Mean rainfall per station-day (mm) at each station by monsoon season, 2017–2024; $r$ is the correlation of the station's rate with its longitude. The west–east gradient is strongest in the south-west monsoon. *Made by* `src/plot_task1.py from src/task1.py`.

### 4.2 Task 1: changes across 2017–2024

Month-balanced annual rainfall per station (Figure 8) was lowest in 2019 (1,742 mm) and highest in 2021 and 2022 (2,921 and 2,907 mm), against a 2017–2024 mean of 2,588 mm. We balance by month and divide by station because a plain network sum exaggerates the change. It puts 2021 at 1.58 times 2017; per station

the ratio is 1.18. The difference is that more stations reported in 2021. Restricting the calculation to the fixed panel of 32 stations that reported in every year changes no annual total by more than 1.5 %, so changes in which stations reported do not explain these totals.

Year-to-year differences follow how often it rained much more than how hard. Across the eight years, the annual total correlates at 0.95 with the frequency of rain and at 0.48 with its intensity. In 2019 the frequency index fell to 67 (2017–2024 mean = 100) while intensity stayed at 98. From 2020 on, frequency stayed above the mean, reaching 111 in 2023.

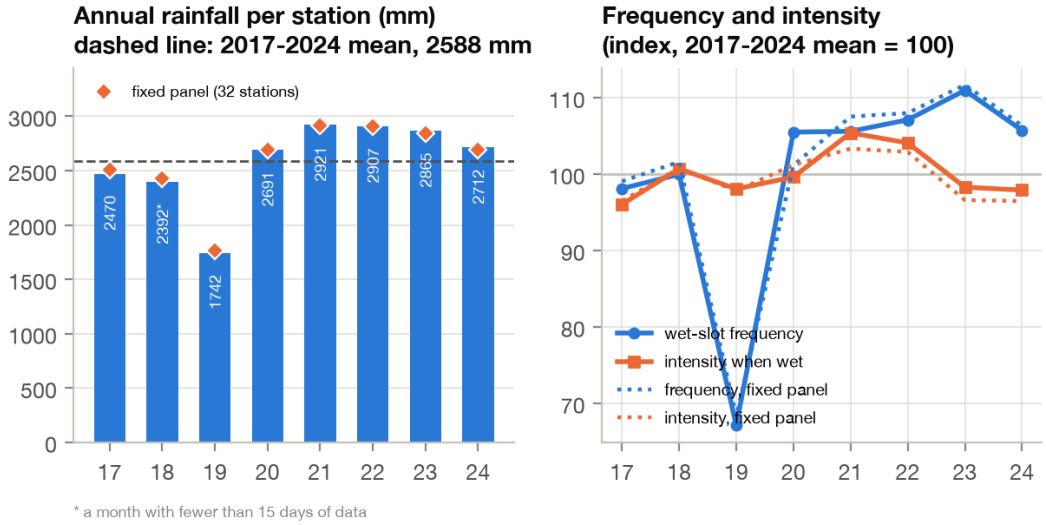



**Figure 8:** Left: month-balanced annual rainfall per station (mm), 2017–2024, for all stations (bars) and the fixed panel of 32 stations (diamonds). Right: frequency of wet slots and intensity when wet, as indices with the 2017–2024 mean at 100, for all stations (solid) and the fixed panel (dotted). The 2019 dip is a drop in frequency, not intensity, and the panel tracks the full network in every year. *Made by* `src/plot_task1.py from src/task1.py`.

The seasonal anomalies (Figure 9) show which part of the year made each year wet or dry. In 2019 every season was dry, the south-west monsoon most of all (−49 %). The south-west monsoons of 2020 and 2021 were wet (+35 % and +36 %), as were the October–November inter-monsoon of 2022 (+47 %) and the north-east monsoon of 2023 (+55 %). Single months (Figure 10) swing much more, and some of those cells are distorted by gaps (Section 4.3).

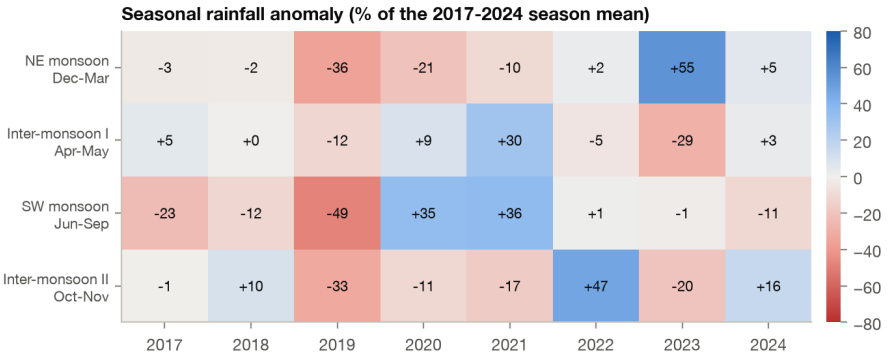



**Figure 9:** Seasonal rainfall anomaly for each year 2017–2024, as % of the 2017–2024 mean for that season (blue wetter, red drier). 2019 is dry in all four seasons; the largest wet anomaly is the 2023 north-east monsoon. *Made by* `src/plot_task1.py from src/task1.py`.

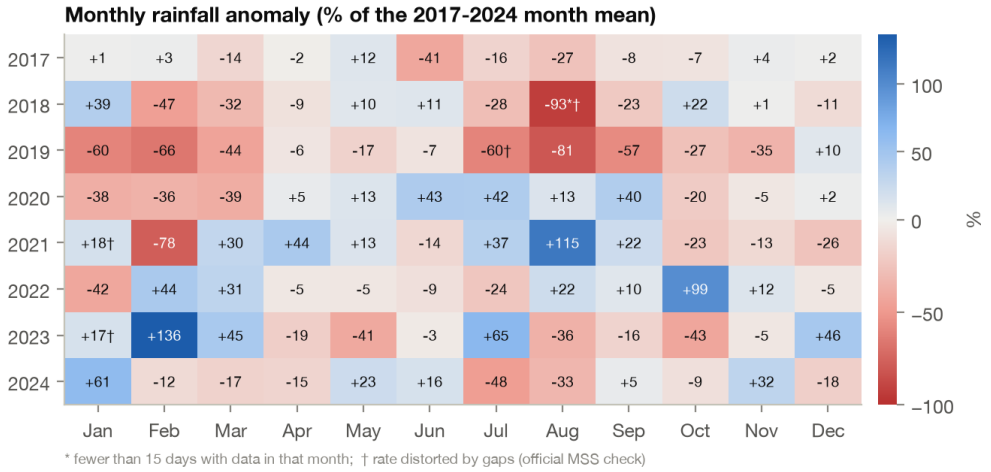



**Figure 10:** Monthly rainfall anomaly for each year 2017–2024, as % of the 2017–2024 mean for that calendar month. * fewer than 15 days of data; † the comparison with official MSS records finds our rate distorted by gaps (Table 7). The marked cells should not be read at face value. *Made by* `src/plot_task1.py from src/task1.py and src/mss_validate.py`.

Heavy rain was rarest in 2019 and most frequent in 2021 and 2022 (Figure 11). Hours with at least 20 mm numbered 19.9 per 10,000 complete station-hours in 2019, against 28.4 in 2017 and 34.3 in 2018, and 39.0 in 2022. Days with at least 50 mm numbered 15.3 per 1,000 complete station-days in 2019, against 21.7 and 24.0 in the two years before, and 35.1 in 2021. Across stations, the median of each station's annual maximum hour rose from 53.6 mm in 2017 to 63.6 mm in 2022, and the largest single hour in the data (140.2 mm) fell in 2022. The official MSS records show the same jump on the days both sources cover completely: 16.3 heavy days per 1,000 matched station-days in 2019 and 35.1 in 2021 (ours: 15.7 and 34.4), so it is not an artefact of our data. We do not call it a trend. Eight years is a short record, and the years with the most heavy rain were also the wettest overall.

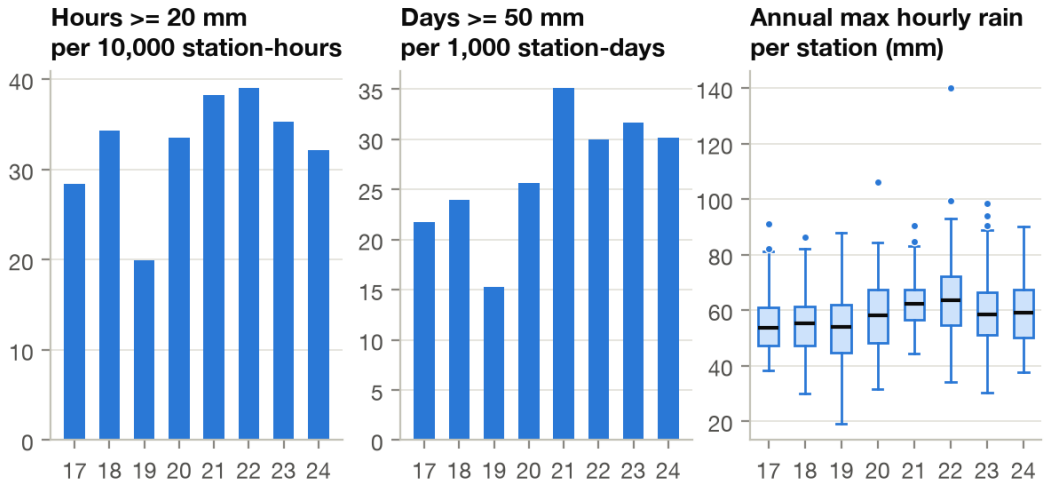



**Figure 11:** Rain extremes by year, 2017–2024: hours with at least 20 mm per 10,000 complete station-hours (left), days with at least 50 mm per 1,000 complete station-days (middle), and the spread across stations of each station's largest hourly total (right). Both counts are lowest in 2019 and highest in 2021–2022. *Made by* `src/plot_task1.py from src/task1.py`.

### 4.3 Task 1: months that the gaps distort

A rate computed over reported slots is unbiased only if the missing days are like the reported ones. To find the months where they are not, we compared each year-month's rate with the official MSS rate on the same station-months (Table 7). In most months the two agree (median ratio 1.00). Where they disagree, the month has large gaps.

January 2021 is the clearest case. Figure 10 shows it only +18 % above an average January, yet MSS reported

it as Singapore's wettest January on record. On the same stations the official rate is 13.38 mm/day against our 9.18 (ratio 0.69). The network was down on 1–4 January (from 07:55 on 1 January), and those were the wettest days of the month. Measured against our average January, the official figure would be +70 %. Our 2021 annual total (2,921 mm) is too low for the same reason.

August 2018, with data on only 3 days, shows as −93 % in Figure 10. That is the gap, not the weather: on the same stations the official rate is 2.92 mm/day against our 0.46.

January 2023 is distorted the other way. With 23 days of data network-wide, our rate is 1.25 times the official one on the same station-months, because the days we lack were drier than the days we have. Figure 10 therefore shows the month as wetter than usual (+17 %), while officially it was drier (−7 %). Changi (S24) is missing for most of the month (Table 4).

The two rates also differ by more than 15 % in 2 months without a large gap: Feb 2018 (1.44), Aug 2020 (1.28). Partial days and the different spread of the two networks over the island are the likely causes. The fixed-panel check covers the annual totals and the frequency and intensity indices; any claim about a single month should be read together with Table 7.

| Month | Ours (mm/day) | Official (mm/day) | Ratio | Official rain on days we lack | Our anomaly (%) |
|---|---|---|---|---|---|
| Aug 2018 | 0.46 | 2.92 | 0.16 | 99 % | −93 |
| Jan 2021 | 9.18 | 13.38 | 0.69 | 32 % | +18 |
| Jul 2019 | 1.99 | 2.38 | 0.84 | 30 % | −60 |
| Jan 2023 | 9.11 | 7.30 | 1.25 | 18 % | +17 |

**Table 7:** Months whose rate differs by more than 15 % from the official rate on the same station-months while more than 10 % of the official rain fell on days we lack. A ratio below one means our rate is too low. *Made by* `src/mss_validate.py`.

### 4.4 Task 2: model choice on 2023

Table 8 gives every candidate and both baselines on the validation year, pooled over the threshold grid (base rate 0.0230).

| Model (validation 2023) | AUC-ROC | AUC-PR | Brier | BSS | Log loss | Mean $p$ | $p$ rises with $\theta$ | Fit (s) |
|---|---|---|---|---|---|---|---|---|
| Climatology baseline | 0.788 | 0.094 | 0.02165 | 0.037 | 0.0995 | 0.0211 | 0.00 % | – |
| Persistence baseline | 0.868 | 0.422 | 0.01528 | 0.321 | 0.0682 | 0.0220 | 0.00 % | – |
| A: logistic regression (reg 0.0001) | 0.797 | 0.082 | 0.02175 | 0.033 | 0.0960 | 0.0210 | 0.00 % | 6 |
| A: logistic regression (chosen) | 0.797 | 0.085 | 0.02174 | 0.033 | 0.0959 | 0.0211 | 0.00 % | 6 |
| A: gradient-boosted trees (depth 5, trees 60) | 0.808 | 0.098 | 0.02179 | 0.031 | 0.1007 | 0.0325 | 0.04 % | 121 |
| B: logistic regression (chosen) | 0.963 | 0.668 | 0.01204 | 0.465 | 0.0477 | 0.0228 | 0.00 % | 15 |
| B: logistic regression (reg 0.001) | 0.964 | 0.663 | 0.01214 | 0.460 | 0.0479 | 0.0228 | 0.00 % | 12 |
| B: gradient-boosted trees (depth 5, trees 80) | 0.978 | 0.728 | 0.01100 | 0.511 | 0.0510 | 0.0352 | 0.66 % | 167 |
| B: gradient-boosted trees (depth 7, trees 80) | 0.980 | 0.732 | 0.01092 | 0.514 | 0.0506 | 0.0358 | 1.57 % | 286 |

**Table 8:** Validation year 2023: every candidate and both baselines, weighted, pooled over $\theta$. BSS: Brier skill score against the base rate. "$p$ rises with $\theta$": share of adjacent-$\theta$ pairs where the predicted probability goes up, which should never happen. The selection rule takes the lowest log loss. *Made by* `src/task2_train.py`.

The two baselines are simple rules. Climatology predicts the training-years frequency of the event for that station, month and hour; persistence assumes the last half hour repeats. Model A is no better than climatology: AUC-PR 0.085 against 0.094, and Brier skill 0.033 against 0.037. Given only the station, the time and $\theta$, the best possible answer is the climatological frequency, and MLlib learns roughly that. Model B is well ahead of both baselines. Its AUC-PR is 0.668, against 0.422 for persistence and 0.094 for climatology, and its Brier skill 0.465, against 0.321 and 0.037.

The gradient-boosted version of B ranks events better: AUC-PR 0.732 against 0.668 for the logistic regression, and Brier score 0.01092 against 0.01204. Our selection rule, written into `config/eval_plan.json` before any model was fitted, takes the lowest validation log loss, and that is logistic regression (0.0477 against 0.0506). We kept the rule's choice. Switching to the trees after seeing the 2023 results would have made the validation year part of the model search, which is what fixing the rule in advance was meant to prevent.

The trees lose on calibration (Figure 12). They over-predict, more so as $\theta$ rises: at $\theta = 20$ mm their mean prediction is 0.0154 where the event rate is 0.0011. For 1.6 % of adjacent threshold pairs they also give a

*higher* probability to the higher threshold, which cannot be right. Logistic regression is monotone in $\theta$ by construction here and never does this. We did not evaluate the trees on 2024.

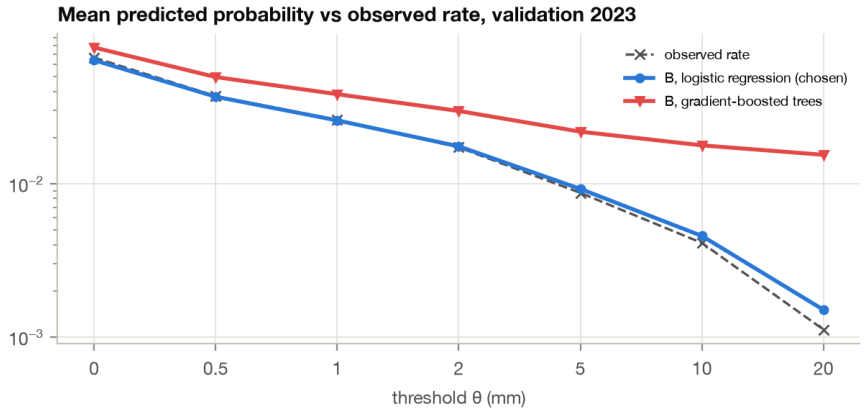



**Figure 12:** Validation 2023: mean predicted probability and observed event rate at each threshold $\theta$ (log scale), for the chosen Model B (logistic regression) and its gradient-boosted alternative. The logistic regression follows the observed rate; the trees over-predict, and more so at higher $\theta$. *Made by* `src/plot_task2.py from src/task2_train.py`.

### 4.5 Task 2: the test year 2024 (evaluated once)

We ran the test once, at 2026-10-17 15:05, with the plan unchanged since it was fixed and the chosen models loaded from disk without refitting. Table 9 gives the pooled results and Table 10 the results per threshold.

| Model (test 2024, once) | AUC-ROC | AUC-PR | Brier | BSS | Log loss | Mean $p$ | $p$ rises with $\theta$ |
|---|---|---|---|---|---|---|---|
| Climatology baseline | 0.798 | 0.085 | 0.02095 | 0.035 | 0.0950 | 0.0212 | 0.00 % |
| Persistence baseline | 0.865 | 0.404 | 0.01507 | 0.305 | 0.0674 | 0.0216 | 0.00 % |
| A: logistic regression | 0.806 | 0.084 | 0.02096 | 0.034 | 0.0925 | 0.0212 | 0.00 % |
| B: logistic regression | 0.959 | 0.654 | 0.01186 | 0.453 | 0.0478 | 0.0219 | 0.00 % |

**Table 9:** Test year 2024, evaluated once: the chosen models and both baselines, weighted, pooled over $\theta$. *Made by* `src/task2_test.py`.

| $\theta$ (mm) | Base rate | Clim. AUC-PR | BSS | Pers. AUC-PR | BSS | A AUC-PR | BSS | B AUC-PR | BSS |
|---|---|---|---|---|---|---|---|---|---|
| all | 0.0222 | 0.085 | 0.035 | 0.404 | 0.305 | 0.084 | 0.034 | 0.654 | 0.453 |
| 0 | 0.0635 | 0.119 | 0.024 | 0.522 | 0.362 | 0.115 | 0.022 | 0.740 | 0.500 |
| 0.5 | 0.0362 | 0.072 | 0.016 | 0.464 | 0.317 | 0.071 | 0.015 | 0.705 | 0.493 |
| 1 | 0.0256 | 0.052 | 0.011 | 0.407 | 0.263 | 0.052 | 0.011 | 0.640 | 0.435 |
| 2 | 0.0169 | 0.036 | 0.007 | 0.331 | 0.194 | 0.036 | 0.008 | 0.547 | 0.343 |
| 5 | 0.0084 | 0.019 | 0.003 | 0.229 | 0.110 | 0.020 | 0.005 | 0.427 | 0.236 |
| 10 | 0.0038 | 0.010 | -0.000 | 0.151 | 0.057 | 0.010 | 0.002 | 0.330 | 0.150 |
| 20 | 0.0010 | 0.003 | -0.002 | 0.063 | 0.012 | 0.004 | 0.000 | 0.192 | -0.008 |

**Table 10:** Test 2024 per threshold: AUC-PR and Brier skill score (BSS) of the baselines and the chosen models. B's AUC-PR stays far above the base rate at every threshold, but its BSS falls to about zero at the top one. *Made by* `src/task2_test.py`.

Model B's validation results held on the new year. On 2024 its AUC-PR is 0.654 (validation 0.668), against 0.404 for persistence and 0.085 for climatology. Its Brier skill is 0.453 (validation 0.465), against 0.305 for persistence and 0.035 for climatology. Model A again sits at the level of climatology: AUC-PR 0.084 against 0.085, Brier skill 0.034 against 0.035.

B's skill falls as the event gets rarer (Table 10, Figure 13). Its AUC-PR is 0.740 for any rain, 0.427 for more than 5 mm and 0.192 for more than 20 mm, against base rates of 0.0635, 0.0084 and 0.0010. At the top threshold B still ranks the downpours far above the base rate and above persistence (AUC-PR 0.063), but its Brier skill is about zero (-0.008, against 0.012 for persistence): the probabilities it gives are no more accurate than the base rate itself, and slightly less accurate than persistence. A downpour within half an hour is hard to foresee from rain gauges alone.

Pooled over thresholds, B's probabilities are close to calibrated (Figure 14). In no reliability bin does the mean prediction differ from the observed frequency by more than 0.032.

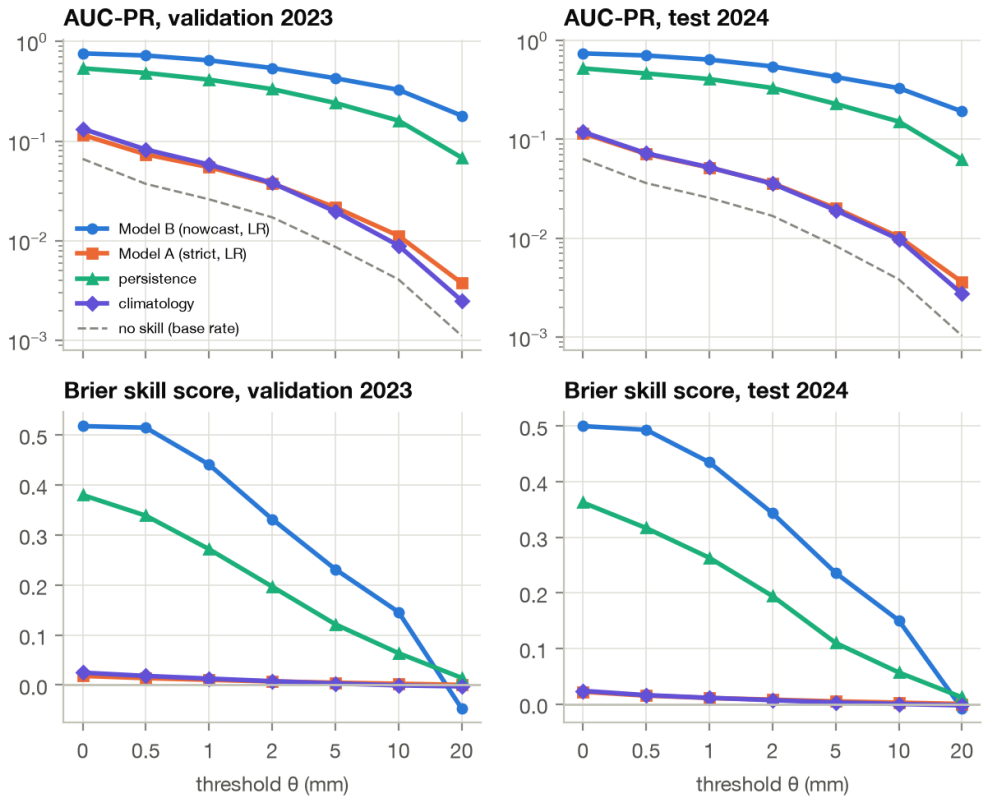



**Figure 13:** AUC-PR (top, log scale; dashed line: the base rate, i.e. no skill) and Brier skill score (bottom) at each threshold, validation 2023 (left) and test 2024 (right). The two years look alike. B leads on AUC-PR at every threshold, while its Brier skill drops to about zero at 20 mm. *Made by* `src/plot_task2.py from src/task2_train.py and src/task2_test.py`.

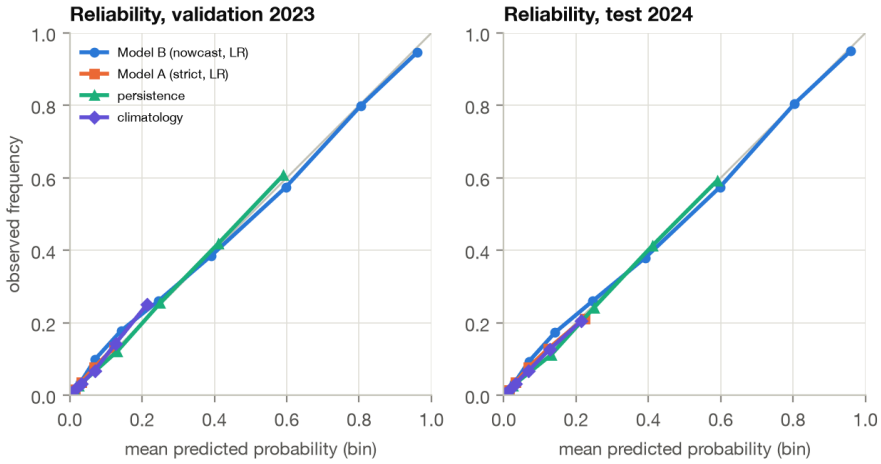



**Figure 14:** Reliability, validation 2023 (left) and test 2024 (right): observed frequency against mean predicted probability per bin, pooled over thresholds (bins with at least 50 rows). A calibrated model lies on the diagonal; B stays close to it up to the highest bin. *Made by* `src/plot_task2.py`.

Figure 15 shows what Model B relies on, as standardised coefficients: the change in log-odds for a one-standard-deviation change in each input. Apart from the threshold terms, which set how the probability falls with $\theta$, the largest are

- the time since the station was last wet ($-0.45$);
- the island-wide maximum 30-minute total ($+0.44$);
- the island-wide share of stations that were wet in the last 30 minutes ($+0.28$), with the share wet at $t$ close behind;

- the station's rain in the last five minutes (+0.26).

Rain at the station and across the island up to $t$ is what makes the next half hour predictable. The hour and day-of-year terms are much smaller once these are known.

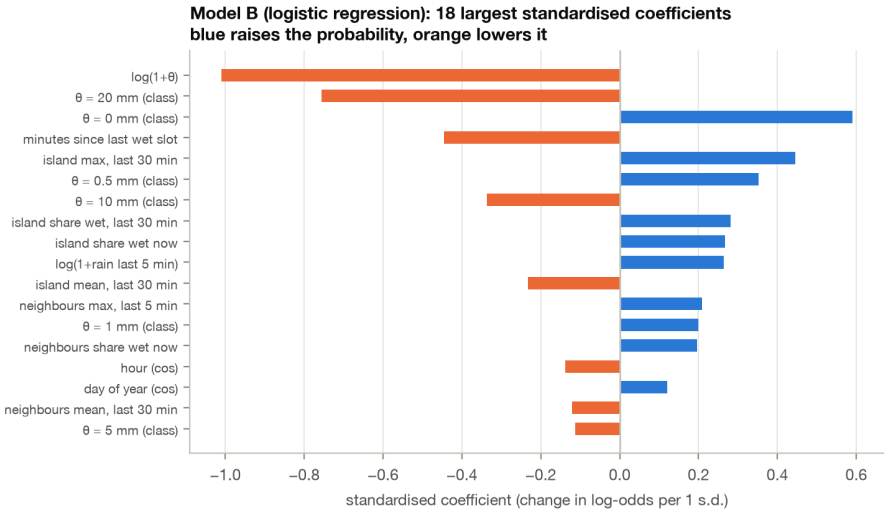



**Figure 15:** Model B (logistic regression): the 18 largest standardised coefficients, as change in log-odds per standard deviation of the input (blue raises the probability, orange lowers it). After the threshold terms, recent rain at the station and across the island dominates; the calendar terms are small. *Made by* `src/plot_task2.py` *from* `src/task2_coefficients.py`.

## 5 Declaration of AI use

We used Claude Code (Anthropic, Claude Opus 5.5) as a coordinating session and a delegated builder session to plan the work, write and run the Spark ingestion, cleaning, analysis and MLlib code, run the experiments, verify the data against official records and draft this report. We directed the work and are responsible for the content and quality of this submission.

## 6 Additional work, observations and limitations

### 6.1 Additional work

- We checked the dataset against the official MSS daily station records (Section 1.3). This is how we found the January 2021 outage and the months in Table 7.
- We recomputed the annual totals and the frequency and intensity indices on a fixed panel of 32 stations, to separate changes in rainfall from changes in the network (Figure 8).
- Before relying on each pipeline check, we fed it a deliberately broken input and confirmed that it stopped the run (`checks/qualify.py`): a missing year, one unaccounted row in the cleaning ledger, a random split, an edited analysis plan, a second test run, a 2024 row in the fitting table, and a hand-typed number in this report. The log is `outputs/qualification/qualification.log`.
- Before the single 2024 run, we ran the test script on 2023. It rebuilt the validation sample row for row, so we knew the test path worked before it touched 2024.

### 6.2 Limitations

1. *Gaps are not random.* A network outage on 1–4 January 2021 removed the wettest days of a record month, and single stations fall silent for weeks at a time. Rates over reported slots cannot recover rain that was never recorded, so the months in Table 7 are misstated: most read too dry, and January 2023 too wet.
2. *No quality control in the source.* Some complete days are simply wrong: a gauge reports zero through a storm, or reports a day of rain that never fell (S36 on 2021-04-01). In all, 101 station-days differ from MSS by more than 20 mm. Every value on such a day is plausible on its own, so our rules cannot flag the day without an outside reference. These days remain in both tasks, but there are too few of them to change the extreme-rain counts (Section 1.3).

3. *Measurement change in 2020.* The share of positive readings on the 0.2 mm grid drops from 98 % in 2019 to 66 % in 2021. The fixed panel holds the stations constant, not the instruments, so it cannot show whether this change moved the across-year results. Statistics of small amounts, such as the share of very light rain, are the most likely to be affected.
4. *Eight years.* The across-year differences (a dry 2019, wet years in 2020–2022) are in the data, but eight years cannot separate a trend from natural variability such as ENSO and the Indian Ocean Dipole.
5. *Model A is limited by its inputs, not by MLlib.* Under the strict reading of the brief, the station, the time and the threshold carry little information about the next half hour, so Model A ends up as a calibrated climatology.
6. *Model B needs live data.* It uses the station's and its neighbours' readings up to $t$, so it only works where those arrive in real time. If the station has no reading at $t$, `predict.py` falls back to Model A.
7. *Sampling.* The models were fitted on a weighted sample of prediction times (Section 2.4), not on all 34,540,615 labelled training times. The trees in particular might gain from more rows; we did not test this.
8. *Calibration of the trees.* A calibration step fitted on 2023 might remove the trees' over-prediction and let them beat logistic regression on log loss as well. We did not try it: the plan fixed before fitting had no such step, and adding one after seeing 2023 would have meant tuning on the validation year.
9. *Cloud security.* The Dataproc nodes had public IP addresses, which the cluster needed to download the data from data.gov.sg itself, and Google warned that the project's default network firewall allowed ingress from any address. The clusters existed only for this run and were deleted afterwards, with a 30-minute idle limit as a backstop.

## 7 Submission contents

- `README.md`: setup, every stage in order with its runtime, how to load and use the trained models, the Dataproc script and the checks.
- `src/` (pipeline), `checks/` (the checks that stop the pipeline on bad input), `config/` (the analysis plan fixed before fitting, and the record of the single test run), `cloud/` (Dataproc).
- The trained models: `models/model_A` and `models/model_B`, saved with `PipelineModel.save`. Use `src/predict.py --station S24 --time 2024-06-01T15:00 --threshold 0.5 5`.
- This report, built with Tectonic from `report/`, with every number generated into `report/generated/`.